

R3.L01 : Représentations et codages des images

BUT Informatique 3^e semestre

Eric REMY
`eric.remy@univ-amu.fr`

IUT d'Aix-Marseille, site d'Arles

Version du 27 août 2026 à 11:51

amU Aix
Marseille
Université



Pour la meilleure utilisation de ce support de cours, veuillez noter que :

- Un sommaire détaillé est disponible dans le PDF alors pensez à l'utiliser !
- En cliquant sur «  » ou «  » vous accédez à la référence de la fonctionnalité C++ citée sur <http://www.cplusplus.com/> ou <http://en.cppreference.com/> respectivement.
- Toutes les remarques visant à améliorer ce document et le cours en général sont bienvenues.

Ce module va être très intensif ! Seulement 5 semaines : il va vous falloir travailler vite et plus !

Objectif :

- Le but de ce module est de savoir lire et écrire des formats d'images.
- Plus généralement, il s'agit de comprendre et savoir utiliser un format de fichier non-textuel.

Étapes :

- Rappels : fichiers de texte (*plain text*) $\neq$ fichiers binaires (*binary file*)
- Rappels : les données binaires en mémoire et sur disque
- Généralités sur les fichiers d'images
- Études de certains formats

Justification :

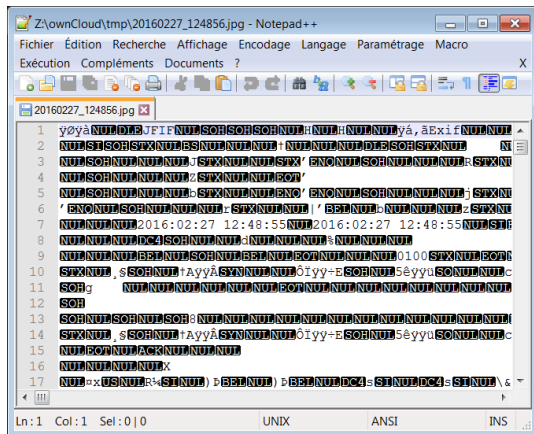
- Les fichiers textuels (XML, HTML, JSON, etc.) sont plus *portables*.
- Les fichiers binaires sont plus *efficaces* (taille, vitesse).

- Un fichier « texte », parfois appelé « texte pur » (US : « plain text ») est simplement une suite d'octets dont chacun représente une lettre ou un code de contrôle (saut de ligne, retour au début de ligne, tabulation, etc.) d'après le codage [ASCII](#) (American Standard Code for Information Interchange).
- Le standard ASCII (1975) étant seulement sur 7 bits, il n'y a **que** 128 codes standardisés...
- ... dont aucun ne comporte les lettres accentuées du français (par exemple).
- Les 128 *autres* codes libres ont longtemps été utilisés pour définir des ensembles **différents et incompatibles** de caractères nécessaires à différents groupes de langues (europe occidentale, grec, cyrillique, arabe, hébreu, etc.).
⇒ tous nos fréquents « problèmes d'accents » viennent de cette période...
- Attention aux conventions pour le saut de ligne :
 - LF (10) : sur tous les UNIX et dérivés (Linux, FreeBSD, OpenBSD, BeOS, MacOSX, AmigaOS, RiscOS, etc.);
 - CR (13) : de l'Apple][à MacOS 9;
 - CR,LF (10,13) : sur tous les héritiers de CP/M, MS-DOS, OS/2, Windows

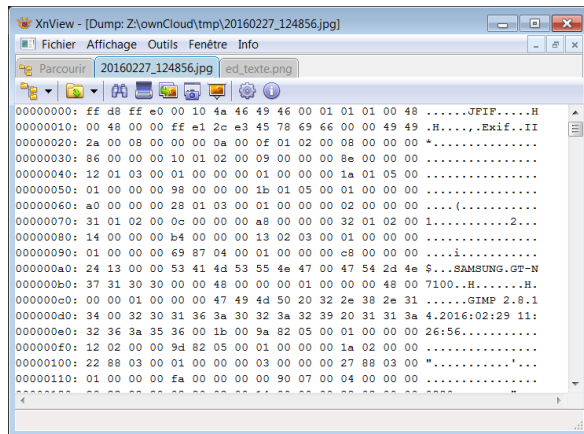
- Le codage UNICODE vise à permettre l'expression uniformisée de toutes les langues et alphabets dans un même document.
- Chaque caractère ou symbole (parmi actuellement ≈ 245000) correspond à un « point de code » numérique (*us : code point*) exprimé sur 32 bits.
- UNICODE existe en plusieurs variantes :
 - UTF-32 : un seul groupe, appelé « codet », de 32 bits (4 octets) par symbole ;
 - UTF-16 : un à deux codets de 16 bits par symbole suivant le nombre de bits nécessaires à exprimer ce point de code (utilisé par Windows, UEFI) ;
 - UTF-8 : 1, 2, 3 ou 4 codets de 8 bits par symbole (Linux, MacOSX, pages Web,... pratiquement tout le monde en vérité!).
- Défaut : En UTF-8 et UTF-16, il n'y a aucun rapport entre la taille d'une chaîne en mémoire (en nombre d'octets) et son nombre de symboles !
- Avantage de l'UTF-8 : les 127 premiers points de code sont *exactement* les codes ASCII !
⇒ compatibilité ascendante
- Plein de subtilités (présence facultative du BOM, sens d'écriture, etc.).
- C'est l'avenir... mais l'ASCII (étendu ou non) a encore la vie dure !

Exemple de fichier « binaire »

- Un fichier qui n'est pas textuel au sens des dernières diapositives est un « fichier binaire ».



Notepad++ window showing a corrupted text file named 20160227_124856.jpg. The text is mostly garbled with non-printable characters and some recognizable strings like 'JFIF', 'Exif', and '2016:02:27 12:48:55'.

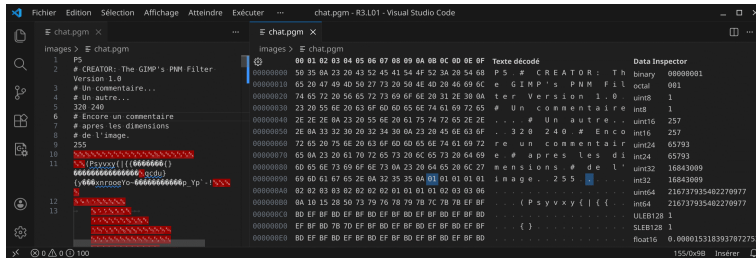


XnView window showing a hex dump of the file 20160227_124856.jpg. The hex dump shows the file's structure, including JFIF and Exif markers, and various metadata fields like 'SAMSUNG.GT-N' and 'GIMP 2.8.1'.

- Ne pas utiliser un éditeur de texte.
- Utiliser un éditeur/visualiseur hexadécimal... et son cerveau !

Exemple de fichier « binaire »

- Un fichier qui n'est pas textuel au sens des dernières diapositives est un « fichier binaire ».



The screenshot shows the Visual Studio Code editor with a file named 'chat.pgm' open. The left sidebar displays the file's metadata, including its format (P5), creator (The GIMP's PWM Filter), version (1.0), dimensions (320x240), and other details. The main editor area shows the raw binary data in hexadecimal, with a corresponding decoded ASCII view on the right. The decoded text is mostly garbled, indicating that the file is not a standard text file but a binary one.

- Ne pas utiliser un éditeur de texte.
- Utiliser un éditeur/visualiseur hexadécimal... et son cerveau !

Vous **ne pouvez pas** espérer le lire/écrire avec les techniques habituelles dédiées aux fichiers de texte car :

- De toutes façons, ce que vous voulez lire, **ce ne sont pas des codes ASCII** : la valeur 4779_{10} ($= 12AB_{16}$) s'écrit (sur 32 bits little endian) avec la suite de 4 octets « AB 12 00 00 » qui n'a rien à voir avec la chaîne de caractères « "4779" » dont les codes ASCII seraient « 34 37 37 39 00 » (5 octets avec le '0' terminal) !
- Rappel : en lecture, l'opérateur « >> » saute (en général) tous les **blancs** (espace, tabulation, retour chariot, nouvelle ligne, etc.)
- Rappel : en écriture, l'instruction « `cout << endl;` » ne produit pas la même séquence d'octets sur toutes les machines (cf. diapo 4).
- Conclusion : vous **ne pouvez pas** imaginer utiliser ces opérations (pourtant bien connues...) si vous souhaitez lire des octets, des mots 16 bits, des mots 32 bits, etc.

Ouverture d'un fichier binaire

- Pour ouvrir en lecture ou écriture un fichier **binaire**, il faut **impérativement** penser à indiquer « `ios::binary` » en deuxième paramètre de l'ouverture sinon vous tomberez dans les pièges de la diapositive précédentes (conversions silencieuses et élimination de certains octets).

Ouverture en lecture ou écriture en binaire

```
1 #include <fstream>
2 ...
3 { // Écrire dans un fichier binaire sur disque
4     std::ofstream ofs("filename.data", std::ios::binary); // Ne pas oublier le deuxième argument !
5     // Mettez ici vos opérations d'écriture vers ofs.
6 } // Vidage du tampon d'écriture vers le disque et fermeture implicite du fichier par le destructeur de ofstream.
7
8 { // Lire depuis un fichier binaire sur disque
9     std::ifstream ifs("filename.data", std::ios::binary); // Ne pas oublier le deuxième argument !
10    // Mettez ici vos opérations de lectures binaires depuis ifs.
11 } // Fermeture implicite du fichier par le destructeur de ifstream.
```

Les types intégraux (et apparentés)

- Chaque type scalaire prend une certaine taille en mémoire.

Types	Taille en octet(s)							Norme ISO
	PC Win				PC Linux		RasPi Linux	
	MS VC++		g++		g++		g++	
	WIN32	WIN64	WIN32	WIN64	ELF32	ELF64	ELF32	
char	1	1	1	1	1	1	1	1
short (int)	2	2	2	2	2	2	2	$1 < \text{sizeof}(\text{short}) \leq \text{sizeof}(\text{int})$
int	4	4	4	4	4	4	4	$4 \leq \text{sizeof}(\text{int})$
long (int)	4	4	4	8	4	8	4	$\text{sizeof}(\text{int}) \leq \text{sizeof}(\text{long})$
long long (int)	8	8	8	8	8	8	8	(Je ne sais pas)
void*	4	8	4	8	4	8	4	
size_t	4	8	4	8	4	8	4	$\text{sizeof}(\text{size_t}) = \text{sizeof}(\text{void}^*)$

- Pas de différence de taille entre « signed » et « unsigned » ; seule l'interprétation des bits change.
- La taille des pointeurs (par définition) dépend du type d'architecture 16, 32 ou 64 bits utilisé pour le bus d'adresse !
- Attention le `long` n'est souvent **pas plus grand** que le `int` !

Types	Taille en octet(s)	Norme
<code>float</code>	4	4 (imposé par IEEE754)
<code>double</code>	8	8 (imposé par IEEE754)
<code>long double</code>	! ?	dépendant de l'architecture !

- Les types « `float` » et « `double` » sont des standards IEEE754 : aucune surprise à craindre.
- Le type « `long double` » existe dans la plupart des compilateurs (sans garantie) pour représenter (en théorie) la taille « naturelle » des données de l'unité de calcul en virgule flottante (FPU) de la machine... qui n'a aucune raison d'être la même partout ! J'ai pu constater :
 - 12 octets sur Linux 32 bits (processeur Atom) ;
 - 16 octets sous Linux 64 bits (processeur i7) ;
 - 8 octets sous Windows 32 ou 64 bits (MS VC++, processeur i7) ;
 - 8 octets sur Raspberry Pi 2 (processeur ARM).

- Nouveauté de C++ 2011 !
- Nécessite « `#include <cstdint>` » 📄 ➕ dans l'en-tête du programme.
- Le compilateur respecte (normalement) la taille indiquée (mais elle pourrait être plus grande).

Taille (octets)	Type (signed)	plage	Type (unsigned)	plage
1	int8_t	$[-128; 127]$	uint8_t	$[0; 255]$
2	int16_t	$[-32768; 32767]$	uint16_t	$[0; 65535]$
4	int32_t	$[-2^{31}; 2^{31} - 1]$ $\approx [-2, 14.10^9; 2, 14.10^9]$	uint32_t	$[0; 2^{32} - 1]$ $\approx [0; 4, 294.10^9]$
8	int64_t	$[-2^{63}; 2^{63} - 1]$ $\approx [-9, 22.10^{18}; 9, 22.10^{18}]$	uint64_t	$[0; 2^{64} - 1]$ $\approx [0; 1, 84.10^{19}]$

- C'est ce qu'on va utiliser en TD et TP.

3 pièges à éviter

- Écrire la valeur d'une variable `u` du type `uint8_t` à l'écran :
 - Faux : « `cout << u ;` » car `u` est un `unsigned char` et va inévitablement afficher une lettre ou (pire !) un code de contrôle !
 - Juste : « `cout << (unsigned int) u;` » qui consiste simplement à convertir en entier la valeur *avant* l'affichage.
- Comparer un `signed` et un `unsigned` (a priori pas de solution miracle) :
 - soit il faut mettre des `unsigned` là où il y aurait dû y en avoir,
 - soit il faut mettre des `signed` partout pour la même raison !
 - Au pire, si les deux types sont justifiés alors convertir explicitement l'un des deux pour faire disparaître le warning... mais savoir qu'on commet ainsi une perte de précision et **faire en sorte qu'elle n'ait pas de conséquence** !
- Les boucles suivantes sont infinies car l'*overflow* ne permet jamais d'atteindre la condition de sortie :
 - `for(uint8_t i=0;i<=255;i++) truc();`
 - `for(uint16_t i=0;i<=65535;i++) truc();`
 - toute autre boucle sur le même principe...

Contraintes d'alignement

- Les données de plus d'un octet doivent être alignées en RAM sous peine de plantage matériel ou de performances dégradées (suivant les processeurs).
- L'adresse doit être multiple de 2 pour les mots 16 bits, multiple de 4 pour les mots 32 bits... (et multiple de 8 pour les mots 64 bits?).
- Remplissage (*us : padding*) dans une classe pour satisfaire ces contraintes.
- Une structure/classe est alignée sur l'exigence de sa plus exigeante donnée membre.
- Conséquence : l'ordre de vos données, que le compilateur *doit impérativement respecter*, peut donc influencer sur la taille totale d'une structure/classe !

Pas aligné

```
1 class noalign { // Alignée multiple de 8
2     char c;      // 1 octet (+7 octets)
3     double d;    // 8 octets
4     int i;       // 4 octets
5     char c2[3];  // 3 octets (+1 octet)
6 }; // Cette structure fait 24 octets.
```

Aligné intelligemment

```
1 class align {
2     double d;    // 8 octets
3     int i;       // 4 octets
4     char c2[3];  // 3 octets
5     char c;      // 1 octet
6 }; // Cette structure fait 16 octets.
```

Little vs. big endian (1/3)

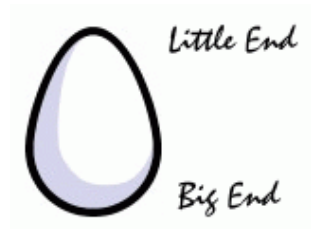
- Pour un même entier 32 bits valant 12345678_{16} , il y a au moins deux façons courantes de l'écrire en mémoire :
 - la façon « petit-boutiste » (*us : little endian*) : commencer par les poids faibles et écrire les octets « 78 56 34 12 » ;
 - la façon « gros-boutiste » (*us : big endian*) : commencer par les poids forts et écrire les octets « 12 34 56 78 ».
- C'est le même principe, avec plus ou moins d'octets, pour un entier 16 bits, un entier 64 bits, etc.

SIMPLY EXPLAINED



Little vs. big endian (2/3)

- Pour votre culture ces appellations amusantes viennent du livre *Les Voyages de Gulliver* (J. Swift, 1721) où une guerre civile éclate au sujet de la façon de manger les œufs à la coque (par le petit ou le gros bout) ;
- Puis d'[un article de Dany Cohen \(1980\)](#) qui se moquait de la querelle d'informaticiens en cours au sujet de l'ordre dans lequel il fallait écrire les octets en mémoire ou sur disque... ou sur le réseau ;
- Les deux sens ont des avantages et des inconvénients... mais pas au point que l'un gagne clairement sur l'autre.
- L'important n'est pas l'ordre lui-même mais bien de se mettre d'accord !
- Remarque : au final, le protocole IP est big endian.



Quelques architectures + systèmes avec leur endianness :

- « petit-boutiste » (*us : little endian*) : Windows ou Linux sur Intel x86 ou Itanium, iOS sur ARM, MacOSX sur Intel,... ;
- « gros-boutiste » (*us : big endian*) : AmigaOS ou MacOS ou Linux sur Motorola PowerPC et 680x0,... ;
- bi-endian (choix au bootstrap puis figé par le système) : ARM, MIPS,...
- (Il existe de *rares* architectures middle endian !... Par exemple, le PDP-11.)

Le **danger**, c'est l'erreur de (non-)conversion lors de la transmission d'une architecture à une autre (par support amovible, par le réseau, etc.).

Écriture octet par octet

Utiliser la méthode « `std::ostream& std::ostream::put(char)` »   :

- Exemple : écriture d'un entier 32 bits little-endian en RAM vers le disque en big-endian.

Écriture octet par octet

```
1 #include <stdint> // Pour int32_t
2 #include <fstream> // Pour ofstream
3 ...
4 {
5     int32_t data = 0x12345678;
6     const char *ptr = (const char*) &data;
7     ofstream os("filename.dat", ios::binary);
8     os.put(*(ptr+3));
9     os.put(*(ptr+2));
10    os.put(*(ptr+1));
11    os.put(*ptr);
12 }
```

- Qualités : très simple à faire, gestion « à la main » de l'endianness.
- Défaut : c'est potentiellement **très lent**.

Lecture octet par octet

Utiliser la méthode « `istream& std::istream::get(char&)` »   :

- Exemple : lecture d'un entier 32 bits big-endian du disque vers la RAM en little-endian.

Lecture octet par octet

```
1 #include <cstdint> // Pour int32_t
2 #include <fstream> // Pour ifstream
3 ...
4 {
5     int32_t data;
6     char *ptr = (char *) &data; // Attention : ici, *ptr n'est pas const !
7     ifstream is("filename.dat", ios::binary);
8     is.get(*(ptr+3));
9     is.get(*(ptr+2));
10    is.get(*(ptr+1));
11    is.get(*ptr);
12 }
```

Écriture par bloc

Utiliser la méthode « `ostream& std::ostream::write(const char*,size_t)` » 📁 © :

- Exemple : écriture d'une structure de 20 octets.

Écriture d'un bloc

```
1 #include <cstdint> // Pour int32_t
2 #include <fstream> // Pour ofstream
3 ...
4 {
5     struct DataBlock { // Aligment : multiple de 4
6         uint8_t  alpha[5]; // 5*1 octets (et 3 de padding)
7         uint32_t bravo[2]; // 2*4 octets
8         uint16_t charlie; // 1*2 octets
9                             // (et 2 de padding)
10    }; // 20 octets au total
11    DataBlock db = {{1,2,4,8,16},{0x01234567,0x89ABCDEF},0x1234};
12    ofstream os("filename.dat",ios::binary);
13    os.write((const char*)&db,sizeof(DataBlock)); // Écriture d'un bloc de 20 octets
14 }
```

- Qualités : très performant (grande vitesse, faible coût de stockage).
- Défaut : ne tient aucun compte de l'endianness ! L'ordre de la mémoire sera reproduit à l'identique dans le fichier.

Utiliser la méthode « `istream& std::istream::read(char*,size_t)` » 📖 C++ :

- Exemple : lecture d'une structure de 20 octets.

Lecture d'un bloc

```
1 #include <cstdint> // Pour int32_t
2 #include <fstream> // Pour ifstream
3 ...
4 {
5     struct DataBlock { // Alignement : multiple de 4
6         uint8_t  alpha[5]; // 5*1 octets (et 3 de padding)
7         uint32_t bravo[2]; // 2*4 octets
8         uint16_t charlie;  // 1*2 octets
9                             // (et 2 de padding)
10    }; // 20 octets au total
11    DataBlock db;
12    ifstream is("filename.dat", ios::binary);
13    is.read((char*)&db, sizeof(DataBlock)); // Lecture d'un bloc de 20 octets
14 }
```

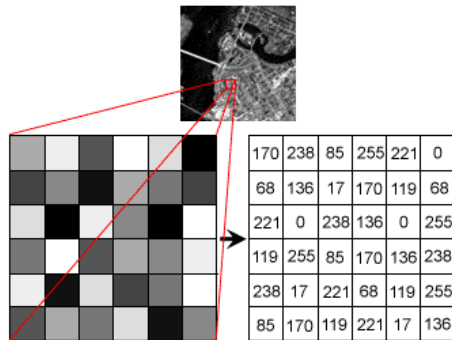
En pratique, on fait comment ?

Pour avoir à la fois *portabilité* et *performance*, il faut combiner le meilleur des deux mondes :

- soit réaliser une sauvegarde « texte » (souvent en XML), éventuellement avec plusieurs fichiers dans une hiérarchie de sous-répertoires, puis en faire une archive « ZIP » (documents Microsoft Office et OpenDocument : .docx, .xlsx, .odt, ods, etc.) ;
- soit réaliser une sauvegarde « binaire » avec lecture/écriture par bloc en s'assurant de respecter un format conventionnel et l'endianness qui y est choisi (JPEG, MPEG, .doc, .xls, etc.).

Images « bitmap » (1/2)

- Dites aussi images « raster »
- Constituées d'une grille de **pixels** (pour l'américain *picture elements*).
- Chaque pixel code l'intensité lumineuse (image en niveau de gris), la couleur (image en couleur), la transparence, etc.
- L'intensité lumineuse est en général codée par une valeur entière entre 0 (noir) et 255 (blanc).
- Pour certains calculs, il est utile de plutôt la considérer réelle entre 0 et 1.



- Principe très général mais gourmand en mémoire : la taille du tableau énumérant les valeurs dépend du **produit** des dimensions largeur $\times$ hauteur .
- Complexité en taille de l'ordre de $O(n^d)$ où n est le côté d'une image carrée et d la dimension de l'espace).

Implantation « à plat » d'un tableau 2D

		0	1	2	3	4	x
0	↓	0	0	10	0	0	
1		10	10	15	10	10	
2	↓	0	0	10	0	0	

(a)

ligne 0					ligne 1					ligne 2				
0	0	10	0	0	10	10	15	10	10	0	0	10	0	0

(b)

- Principe : mettre toutes les lignes bout à bout pour faire un tableau mono-dimensionnel.
- La taille du tableau est de $width \times height$.
- L'accès à un pixel (x, y) de l'image (a) se fait simplement dans le tableau (b)
« `tableau[t]` » à l'indice $t = y \times width + x$.
- Généralisable aux dimensions supérieures :
 $(x, y, z) \mapsto t = (z \times height + y) \times width + x$.

Classe GrayImage : données membres, constructeurs et destructeur

```
1 class GrayImage { // N'oubliez pas <cstdint> !
2     const uint16_t width,height; // Largeur et hauteur de l'image dans [0; 65535].
3     uint8_t *array; // Pointeur vers le tableau alloué dynamiquement des pixels.
4                     // Chaque pixel peut varier dans [0; 255].
5     GrayImage() = delete; // Construction sans paramètre interdite! (C++11)
6 public:
7     GrayImage& operator=(const GrayImage& b) = delete; // Affectation interdite!
8     GrayImage(uint16_t w,uint16_t h); // Construction à la dimension w × h
9     GrayImage(const GrayImage& orig); // Construction par copie
10    ~GrayImage(); // Destructeur
11    ...
12 };
```

- Les dimensions resteront constantes pendant toute la vie de l'instance $\Rightarrow$ pas de ré-allocation du tableau de pixels.
- Interdiction de construire une image sans préciser sa taille ou d'affecter une image à une autre (ce qui changerait sa taille).

Classe GrayImage : implantation des constructeurs et du destructeur

```
1 GrayImage::GrayImage(uint16_t w,uint16_t h) // Construction à partir des dimensions
2   : width(w),height(h),array(nullptr) // Liste d'initialisation des données membres de la classe
3   { array = new uint8_t [width*height]; }
4
5 GrayImage::GrayImage(const GrayImage& o) // Construction de copie
6   : width(o.width),height(o.height),array(nullptr)
7   {
8     array = new uint8_t[o.width*o.height]; // Allocation du nouveau tableau de pixels pour la nouvelle image.
9     for(size_t t=0; t < size_t( width*height ) ; t++)
10       array[t]=o.array[t]; // Recopie des valeurs des pixels de l'image source vers la nouvelle image.
11   }
12
13 GrayImage::~GrayImage()
14 { delete [] array; } // Destruction du tableau de pixels.
```

- La construction implique l'**allocation dynamique** du tableau de pixels car :
 - on ne connaît pas sa taille avant l'exécution et
 - parce que sa (grande) taille sera certainement plus compatible avec un stockage sur le tas (*heap*) que sur la pile (*stack*) !
- On choisit de ne pas initialiser les pixels du tableau... car on le fera avec d'autres fonctions ultérieurement.

Classe GrayImage : accesseurs

```
1 class GrayImage { // N'oubliez pas <cstdint> !
2 public:
3     ...
4     // Consultation de la largeur ou de la hauteur
5     inline const uint16_t& getWidth() const { return width; }
6     inline const uint16_t& getHeight() const { return height; }
7     // Consultation ou modification d'un pixel à partir de ses coordonnées (x, y).
8     inline uint8_t& pixel(uint16_t x, uint16_t y) { return array[y*width+x]; } // Accesseur de modification
9     inline const uint8_t& pixel(uint16_t x, uint16_t y) const { return array[y*width+x]; } // Accesseur de consultation
10    ...
11 };
```

- Déclaration « inline » des accesseurs pour gagner en vitesse.
- Deux variantes de l'accesseur « pixel() » : l'une pour la consultation (« const »), l'autre pour la modification.
- Appel de fonction en Left-value (spécifique au C++ !) grâce au type référence en résultat :

```
1 GrayImage im(320,240); // On crée une image
2 uint8_t gris = im.pixel(10,20); // Consultation en (10,20)
3 im.pixel(10,20) = 255; // Modification en (10,20)
```

- Cette partie sera largement complétée par le module M4202CIN « Connaissances complémentaires en Imagerie Numérique » (système visuel humain, perception de la couleur, etc.).
- L'information de couleur est une quantité **vectorielle** en dimension 3.
- Rappel : un même vecteur peut être exprimé dans plusieurs systèmes différents de repères.
- Il y a donc possibilité d'avoir plusieurs systèmes de représentation des couleurs différents.

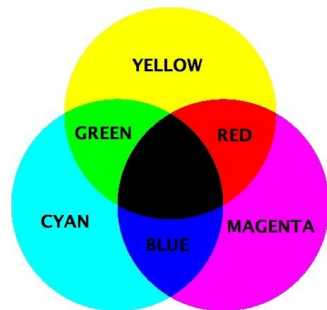
- Rouge - vert / Green - Bleu
- Synthèse additive.
- Ajout de sources lumineuses colorées (par exemple, écran cathodique, triplet de LED colorées, etc.).
- Idée : la pièce est sans lumière, puis on éclaire une table blanche avec des projecteurs teintés. Au centre des trois zones éclairées, la table apparaît effectivement blanche.
- C'est une évidence : sans lumière, il n'y a pas de couleur !



Modèle de couleur « CMY(K) »

- Cyan - Magenta - jaune/Yellow - (noir/blackK)
- Synthèse soustractive.
- Idée : l'ajout d'encre transparente filtrante absorbant une partie de la lumière réémise par du papier (blanc).
- La quatrième composante « blackK » est redondante... mais elle est utile *en pratique* car le mélange des encres n'est jamais assez filtrant pour obtenir un noir satisfaisant.
- Remarque : les couleurs secondaires du modèle CMY sont RGB et vice-versa.
- Conversions : $\forall R, G, B, C, M, Y \in [0; 1]$,

$$\begin{bmatrix} R \\ G \\ B \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} - \begin{bmatrix} C \\ M \\ Y \end{bmatrix} \quad \text{et} \quad \begin{bmatrix} C \\ M \\ Y \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} - \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$

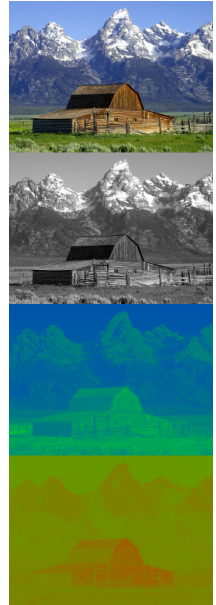


Modèle de couleur « YUV »

- Séparation entre la luminance « Y » (sans couleur) et les deux signaux de chrominance « UV ».
- Modèle utilisé historiquement pour la télévision PAL : le passage à la couleur consistait à ajouter la chrominance au signal préexistant de luminance (télévision noir et blanc).
- Intéressant au niveau codage (JPEG, MPEG, etc.) car la vision humaine sépare la luminance (bâtonnets) de la vision des couleurs (cônes) et qu'elle est plus sensible aux formes en luminance qu'en chrominance.
- $\forall R, G, B, Y \in [0; 1], U \in [-0,436; 0,436], V \in [-0,615; 0,615],$

$$\begin{bmatrix} R \\ G \\ B \end{bmatrix} = \begin{bmatrix} 1 & 0 & 1,13983 \\ 1 & -0,39465 & -0,58060 \\ 1 & 2,03211 & 0 \end{bmatrix} \cdot \begin{bmatrix} Y \\ U \\ V \end{bmatrix} \quad \text{et}$$

$$\begin{bmatrix} Y \\ U \\ V \end{bmatrix} = \begin{bmatrix} 0,299 & 0,587 & 0,114 \\ -0,14713 & -0,28886 & 0,436 \\ 0,615 & -0,51498 & -0,10001 \end{bmatrix} \cdot \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$

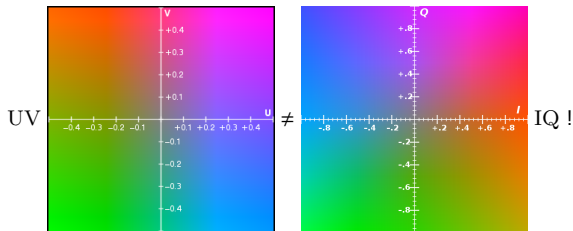


Modèle de couleur « YIQ »

- Même principe que YUV mais avec une décomposition différente de la chrominance.
- Modèle utilisé historiquement pour la télévision NTSC (États-Unis).
- Conversions : $\forall R, G, B, Y \in [0; 1], I, Q \in [-1; 1],$

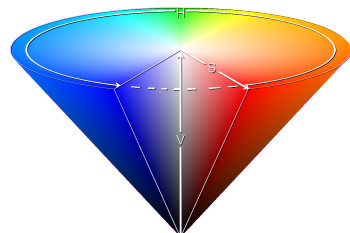
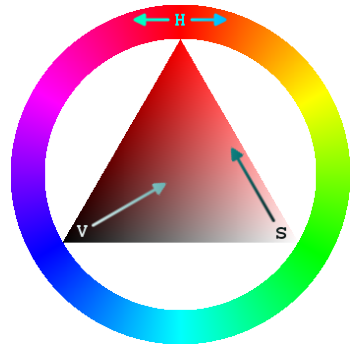
$$\begin{bmatrix} R \\ G \\ B \end{bmatrix} = \begin{bmatrix} 1 & 0,9563 & 0,6210 \\ 1 & -0,2721 & -0,6474 \\ 1 & -1,1070 & 1,7046 \end{bmatrix} \cdot \begin{bmatrix} Y \\ I \\ Q \end{bmatrix} \quad \text{et}$$

$$\begin{bmatrix} Y \\ I \\ Q \end{bmatrix} = \begin{bmatrix} 0,299 & 0,587 & 0,114 \\ 0,595716 & -0,274453 & -0,321263 \\ 0,211456 & -0,522591 & 0,311135 \end{bmatrix} \cdot \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$



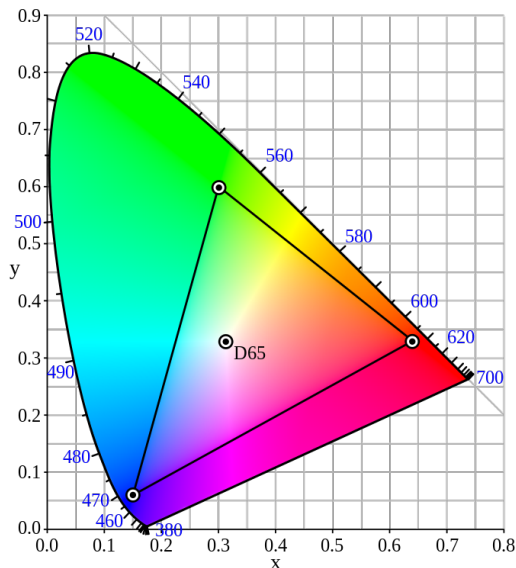
Modèle de couleur « HSV »

- **H**ue/teinte - **S**aturation - **V**alue/valeur
- Proposé en 1978
- Système perceptuel, plus proche de la pratique artistique :
 - ① choisir la teinte de base, puis
 - ② modifier la saturation, c'est-à-dire la quantité de la couleur voulue et, en parallèle,
 - ③ modifier la valeur qui rend plus ou moins sombre.
- La conversion de et vers RGB n'est pas linéaire... mais vous la trouverez facilement, sur [Wikipédia](https://fr.wikipedia.org) par exemple.
- Le système Teinte, Saturation, Lumière (Hue, Saturation, Lightness, HSL est aussi souvent rencontré. Très similaire : double cône au lieu de simple cône.



Autres modèle de couleur

- Beaucoup d'autres systèmes orientés « pro » : sRGB, AdobeRGB, CIE XYZ, CIE xYz, CIE L*a*b*, ...
- Souvent plus complexes (non-linéaires, etc.)
- Meilleure approche de la perception humaine.
- Exemple : représentation des couleurs du sRGB dans le plan xy de l'espace CIE xyY.



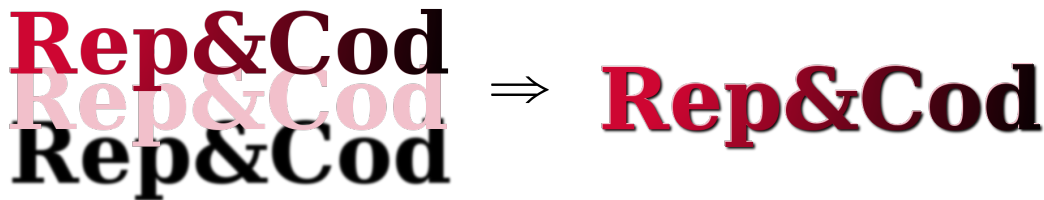
Conception « par pixel » *versus* « par canal »

- Deux modélisations sont possibles :
 - un tableau 2D de pixels qui sont des vecteurs de 3 couleurs (ce que nous allons faire) ou
 - un vecteur de 3 tableaux de pixels représentant une seule composante, appelé **canal** (US : **channel**).
- C'est avant tout une question de point de vue !
- C'est important car on peut souvent faire un traitement sur une image en couleur en l'appliquant simplement à chaque composante séparément... même si ce n'est pas toujours perceptuellement justifié !



« Alpha-channel »

- On peut ajouter aux canaux de couleur, un **canal alpha** (US : **Alpha-channel**) pour coder la transparence d'une image de totalement transparent (0) à complètement opaque (255).
- On trouve souvent la mention « RGBA » : le « RGB » auquel on a ajouté l'alpha-channel.
- La combinaison d'une pile de plusieurs couches d'images comportant une valeur d'alpha-channel s'appelle l'**alpha-blending**.
- Utilisé par exemple pour réaliser les ombres douces sous les fenêtres de vos systèmes d'exploitation préférés.
- Implanté dans tous les logiciels de retouches d'image et, matériellement, dans les cartes graphiques.



- On choisit le modèle RGB (le plus répandu dans les fichiers d'images!).

Classe Color

```
1 class Color {
2 public:
3     uint8_t r,g,b; // Gardez « public » ces trois données dans votre classes.
4     inline Color(uint8_t _r=0,uint8_t _g=0, uint8_t _b=0)
5         : r(_r),g(_g),b(_b)
6     {}
7     friend bool operator==(const Color& c1,const Color& c2); // à implanter !
8     friend Color operator*(double alpha,const Color& color); // à implanter !
9     friend Color operator+(const Color& c1,const Color& c2); // à implanter !
10    ...
11};
```

- On aurait pu en choisir un autre !
- Cette représentation avec 8 bits (soit 1 octet) pour chacune des 3 composantes est souvent appelé « TrueColor », « 24 bits », etc.
- On se contentera au besoin de faire des fonctions de conversions, si nécessaire.
- Les opérateurs « * » et « + » seront nécessaires au ré-échantillonnage bilinéaire.
- Vous devez implanter l'opérateur « == » qui est indispensable à la correction.

- Dupliquer le code de « GrayImage » en remplaçant « uint8_t » par « Color ».

Classe ColorImage

```
1 class ColorImage { // Remplacer Gray par Color
2     uint16_t width,height;
3     Color    *array; // Remplacer int8_t par Color
4     ...
5 };
```

- Rappel : La duplication de code est **néfaste** pour la maintenance !
- Ce serait une bien meilleure idée de :
 - créer un modèle de classe (*template*) « Image<T> » avec tout ce qui est généralisable ;
 - dériver par héritage d'un côté « GrayImage », à partir de « Image<uint8_t> », et de l'autre « ColorImage » à partir de « Image<Color> » ;
 - leur ajouter **seulement** les méthodes *spécifiques* à l'une ou l'autre.
- Comme c'est *plus difficile*, merci de vous entendre pour qu'il n'y ait **pas plus d'un à deux étudiants** (motivés et bons en C++) par groupe de TP à tenter cette expérience !

Image avec palette de couleurs (1/2)

- Le volume de mémoire et/ou le volume des supports de stockage a longtemps été un frein au développement du traitement d'image par un ordinateur.
- Pour gagner de la place en mémoire ou sur disque à l'époque où le stockage *direct* TrueColor n'était pas envisageable, un codage *indirect* des images utilisait une **Palette de couleurs**, c'est-à-dire un tableau de n couleurs RGB.
- Le tableau de pixels ne contient pas la valeur de couleur sous forme d'un triplet RGB mais un indice de couleur dans le tableau de palette !

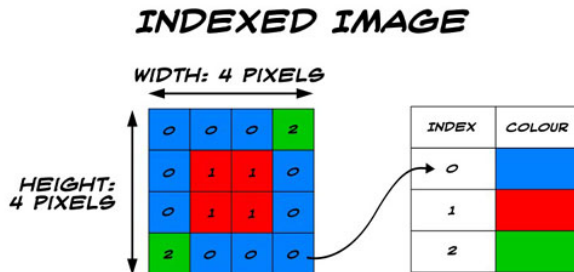


Image avec palette de couleurs (2/2)

Quelques indications de dates et de performances graphiques :

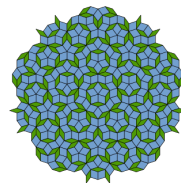
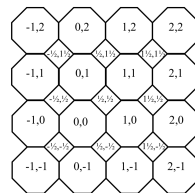
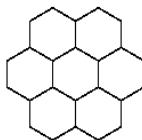
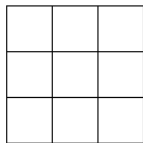
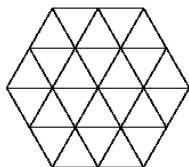
- Apple II (1977) : 280×192 pixels avec 6 couleurs ;
- PC mode CGA (1981) : 320×200 pixels en 4 couleurs parmi 4 palettes de 16 couleurs fixées ;
- Oric-1 ou Atmos (1983) : 240×200 pixels en 2 couleurs parmi 8 couleurs fixées ;
- Amiga chipset OCS (1985) : de 320×256 en 32 couleurs à 640×512 en 16 couleurs parmi 16 millions (8 bits par composante) ;
- PC mode VGA (1987) : 320×240 en 256 couleurs ou 640×480 pixels en 16 couleurs parmi une palette de 262144 (6 bits par composante) ;
- PC mode SuperVGA (≈ 1990) : 640×480 en 256 couleurs ou 640×480 pixels en 16 couleurs parmi 262144(?).



Maillages et pavages (1/3)

Idée : où sont placés les pixels ?

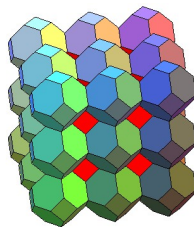
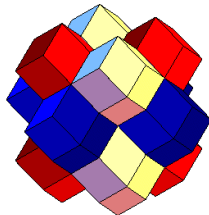
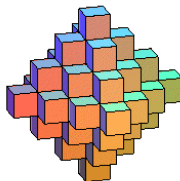
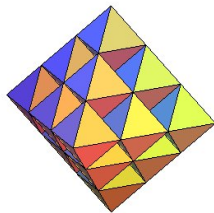
- Il existe des pavages réguliers du plan :
 - triangulaire ;
 - carré (un défaut : deux types de voisins, ceux ayant une arête commune, qui sont plus proches, et ceux qui ont un sommet commun, qui sont $\sqrt{2}$ fois plus loin) ;
 - rectangulaire (identique au précédent si les pixels ont un rapport largeur/hauteur valant 1) ;
 - hexagonal (tous les voisins sont à la même distance du pixel central) ;
 - etc.



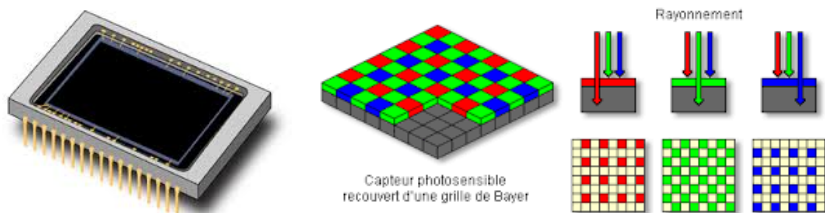
- ... et aussi des pavages non-périodiques : pavage de Penrose (≈ 1970 , quasi-périodique !), de Wang, etc.

Idée : où sont placés les pixels ?

- D'autres maillages existent pour la 3D (mais, en dehors du cubique, c'est plus délicat !) :
 - octaèdre-tétraèdre ;
 - cubique (son défaut : cette fois, trois types de voisins avec, du plus proche au plus loin, les voisins par une face, par une arête, par un sommet) ;
 - pavage par des dodécaèdres rhombiques (maillage cubique à faces centrées) ;
 - pavage par des octaèdres tronqués (maillage cubique centré) ;
 - etc.



- Dans un capteur CCD, à cause de contraintes matérielles (passage des fils de liaison, contraintes de photo-gravure, etc.), il y a en général de la surface perdue (qui ne mesure pas la lumière) entre les photosites.
- Pour les capteurs couleurs, il y a également des disparités entre les sites de mesure de chacune des 3 couleurs : en général, il y a deux fois plus de sites de mesure du vert que du rouge et du bleu.



- 99% du temps, on considère, même si c'est faux, que l'échantillonnage est fait de pixels carrés, sans intervalles et avec une mesure des trois composantes sur chaque pixel...

Densité

- Quelle densité de pixels en fonction de l'aire du capteur, de l'écran, de l'imprimante, etc. ?
- Mesurée en nombre de **points ou de pixels par pouce** (ppp ; *us* : *dpi*) (1 in = 2,54 cm).
- Attention : la densité n'est pas forcément la même horizontalement et verticalement. On n'a alors pas une mesure isotrope (équivalente dans toutes les directions).
- Ordres de grandeurs : ≈ 120 ppp pour un écran TFT, ≈ 4800 ppp pour un scanner A4, de 600 à 1200 ppp pour une imprimante jet d'encre, 2400 ppp pour une laser.

Rapport d'aspect

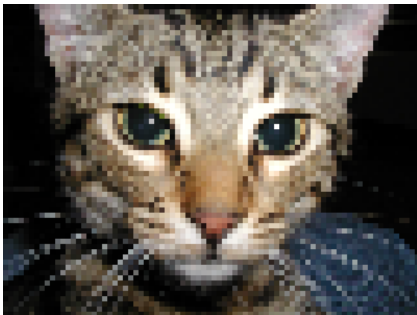
- C'est le rapport entre la taille horizontale et verticale d'un **pixel**.
- Ne pas confondre avec le rapport d'aspect de l'**écran** (4:3, 16:9, 16:10, etc.).
- Si le rapport vaut 1:1 alors une ligne verticale de n pixels et une ligne horizontale de n pixels ont la même longueur à l'écran/l'impression/etc.
- Remarque : rien n'empêche d'avoir une image carrée avec des pixels rectangulaires ou une image rectangulaire avec des pixels carrés ! Ne vous fiez pas à la résolution d'image pour espérer deviner le rapport d'aspect de ses pixels.

Fréquence d'échantillonnage

320×240



80×60



160×120

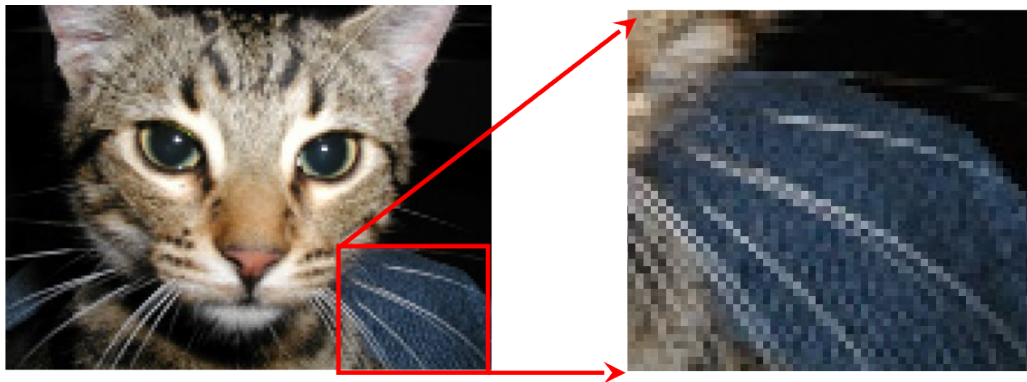


40×30



Aliasing spatial

- Cas où les détails sont plus fins que ceux perçus par le capteur.



- « Marches d'escalier » plus ou moins marquées
- Règle physique : échantillonnage au moins au double de la fréquence (c'est-à-dire à la moitié de la longueur d'onde) du signal à échantillonner !

- Combien de niveaux d'intensité sont disponibles ?
→ influe sur la taille du codage

	Apparence	bits/pixel
Noir & blanc		1
4 niveaux de gris		2
16 niveaux de gris		4
256 niveaux de gris		8
256 couleurs		8
16,7 millions de couleurs		$3 \times 8 = 24$

- Combien sont *réellement* utilisés ?
→ utilisation éventuelle d'une palette (codage indirect)
- Sensibilité du dispositif d'acquisition ?
- Capacités du dispositif de restitution ?
- Capacités de notre système de vision !

Exemples de quantifications (sans tramage)

16,7M



32



128



8



Exemples de quantifications (avec tramage de Floyd-Streinberg)

16,7M



128



32

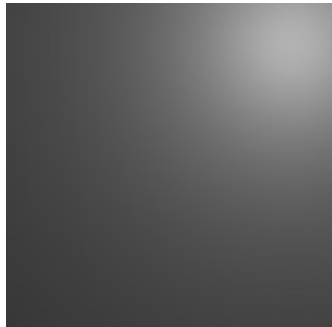


8

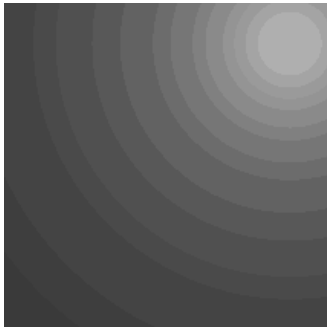


Aliasing colorimétrique : bandes de Mach

- Lorsqu'un dégradé de couleurs ou de gris n'est pas assez détaillé
- On perçoit des zones uniformes séparées appelées bandes de Mach
- Phénomène renforcé par notre système visuel



256 gris



16 gris



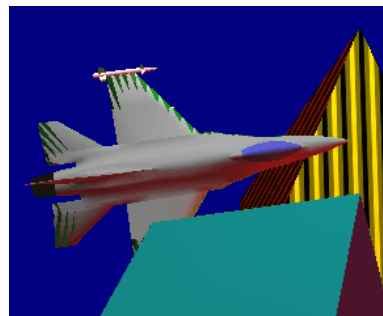
renforcement du contraste

Que faire lorsque les données à représenter ou à étudier sont plus détaillées que ce que l'œil peut percevoir ?

- Exemple n° 1 : en astronomie, plusieurs milliers de niveaux de gris numérisés contre seulement une vingtaine perceptibles par un être humain.
- Exemple n° 2 : en imagerie satellitaire multimodale, plusieurs images prises simultanément dans plein de longueurs d'onde ($\approx$ couleurs) : infrarouge thermique, infrarouge lointain, infrarouge proche, rouge, vert, bleu, ultraviolet, ... , altitude, polarisation, propriétés de surface, etc.
- Solution n° 1 : visualisation en « fausses couleurs »
- Solution n° 2 : fenêtrage

Quantification de la profondeur

- OpenGL utilise plusieurs tampons mémoire (buffer) :
 - tampon de couleur ;
 - tampon de profondeur (Z-buffer) ;
 - ...
- Le Z-buffer est en général sur 16 bits.
- Z-fighting : à cause d'une précision insuffisante dans les calculs, plusieurs objets sont projetés sur la même valeur de Z. Aléatoirement en apparence, ils gagnent l'un sur l'autre leur présence à l'écran... et donnent un effet de hachures sur le tampon de couleur ! L'erreur étant plus liée à la position de la caméra par rapport à la scène qu'à celle de l'objet, les hachures peuvent « bouger » à la surface de l'objet lorsque la caméra change.



Propriétés souhaitées d'un stockage d'image :

- Échange entre deux ordinateurs (ou personnes) ;
- À l'identique (ou avec perte minimale et contrôlée) ;
- Indépendant du stockage en mémoire (en particulier, indépendant de l'architecture matérielle, de l'endianness, etc.) ;
- Rapide à la lecture et si possible à l'écriture ;
- Possibilité de stocker des images en différentes quantifications (N&B, gris, couleur, palette, etc.) ;
- Éventuellement, stockage de plusieurs images dans un seul fichier ;
- Possibilité de rajouter des informations en plus des données purement images (auteur, date, etc.).

- Minimiser la taille sur disque ou transférée par le réseau
- Compression sans perte :
 - Réduire sans abîmer du tout l'image ;
 - Nécessite un sur-coût de calcul ;
 - Vise à supprimer les redondances d'informations.
- Compression avec perte :
 - Perte de qualité volontaire et contrôlée ;
 - Gros gain en taille ;
- Taux de compression

$$\text{Taux de compression} = \frac{\text{Taille originale}}{\text{Taille compressée}}$$

- Exemple

de 100 Ko brut à 20 Ko compressé $\Rightarrow$ Taux de 5 : 1

- Formats simples non-compressés :
 - Niveaux de gris : PGM, TGA, BMP, TIFF,...
 - Couleur : PPM, TGA, BMP, TIFF, ...
- Formats avec compression sans perte :
 - RLE sur niveaux de gris et couleurs : TGA, BMP, TIFF, ...
 - Huffman sur gris ou palette : GIF, TIFF, ...
- Formats compressés avec perte :
 - Gris et couleurs : JPEG (JFIF), JPEG2000, TIFF, ...
- Pour aller plus loin :
 - Wikipédia
 - <http://www.dcs.ed.ac.uk/home/mxr/gfx/>
 - <http://www.fileformat.info/>

- En tête (header)
 - « Magic number » : pour reconnaître le format.
 - Largeur, hauteur.
 - Quantification : couleur, gris, noir & blanc,...
 - (Éventuellement, mode de compression).
 - Présence d'une palette ou non.
 - Autres informations...
- (Éventuellement la palette)
 - Nombre de couleurs de la palette
 - Quantification
 - Liste des couleurs de la palette
- Corps de l'image
 - Suite de pixels, éventuellement compressée
- (Autres informations éventuelles...)

- Portable (Bit/Gray/Pixel) Map $\Rightarrow$ Portable *aNy* Map
- Peu performant et donc peu utilisé
- Extrêmement simple à lire et à écrire...
- Donc très bien pour débiter !
- Codage direct uniquement / Pas de palette.
- 2 variantes :
 - Format ASCII pur : l'image est alors transcrite en un texte.
Codage vraiment peu séduisant mais assurant une portabilité facile.
 - Format ASCII + Binaire : seul l'entête est du texte, les données des pixels sont binaires.
Beaucoup plus intéressant !

Run Length Encoding (RLE)

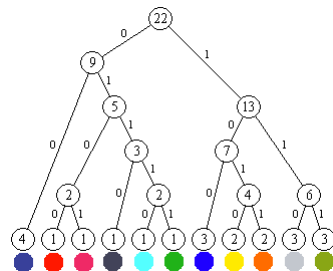
- Compression sans perte
- Efficace lorsque nombre de couleurs limité
- Peu efficace de manière générale mais simple
- Principe :
 - Remplacer une suite de pixels identiques par le nombre de répétitions et la valeur.
→ Réduction
 - Remplacer une suite de pixels différents par la longueur de cette suite (par exemple, en négatif, pour permettre de différencier), et la série de valeurs.
→ Augmentation
- Exemple :

La suite A.B.C.D.D.D.D.D.E.E.E.E.F.G (14 symboles) devient
-3.A.B.C.5.D.4.E.-2.F.G (11 symboles)
- Mis en œuvre dans TARGA (vu en TP) et d'autres

- MS-Windows & IBM-OS/2 bitmap
- Utilisé aussi pour les icônes, curseurs, etc.
- Très comparable à Targa
- Supporte toute combinaison de : N&B, gris, couleurs, avec et sans palette, avec et sans RLE
- Très légèrement plus difficile à manipuler que Targa mais sans nouveauté particulière, donc ne sera vu en TP

Codage de Huffman

- Compression sans perte
- Efficace lorsqu'il y a peu de couleurs différentes
- Plus efficace que le RLE... mais plus difficile
- Principe :
 - 1 Calculer l'histogramme de l'image
 - 2 Construire un arbre binaire en regroupant les couleurs/nœud les moins utilisées par deux. Étiqueter le nœud ainsi formé avec la somme des fréquences des nœuds fils.
 - 3 Recommencer jusqu'à avoir créé la racine avec pour fréquence le nombre total de pixels.
 - 4 Par convention, on étiquette les arrêtes gauches à 0 et les droites à 1.
 - 5 Le code d'une couleur est donné par le chemin partant de la racine et allant jusqu'à la feuille correspondante : par exemple, le rouge est « 0100 ».
- Moins de bits pour les couleurs qui reviennent le plus qu'ite à utiliser plus de bits pour celles qui sont rares



Compression Lempel-Ziv-Welch (LZW)

- Compression sans perte ; efficace pour peu de couleurs
- Algos de Ziv et Lempel (77-78)
- Variante plus rapide de Welch (1983)
- Marche même sur images tramées car cherche la répétition des motifs
- Très efficace mais difficile !
- Brevet UNISYS de 1985... jusqu'à 2003-2004 !
- Principe :
 - Trouver des séquences de pixels qui se répètent dans l'image ;
 - Trier les motifs par ordre de fréquence
 - Coder les motifs les plus fréquents par peu de bits quitte à coder les moins fréquents avec plus de bits.

- Utilise l'algorithme LZW
- Longtemps sous copyright d'UNISYS
- Libre d'utilisation depuis 2004... mais seulement pour GIF89a !
- Les versions plus récentes ne se sont jamais imposées.
- Supporte :
 - Gris 8 bits direct ou bien palette de 256 couleurs parmi 16,7 millions.
 - Animation de quelques dizaines d'images
 - Possibilité d'avoir une palette différente par image
 - Transparence (tout ou rien) en sacrifiant une des couleurs.
- Adapté lorsqu'il y a peu de couleurs : logo, schémas, etc.

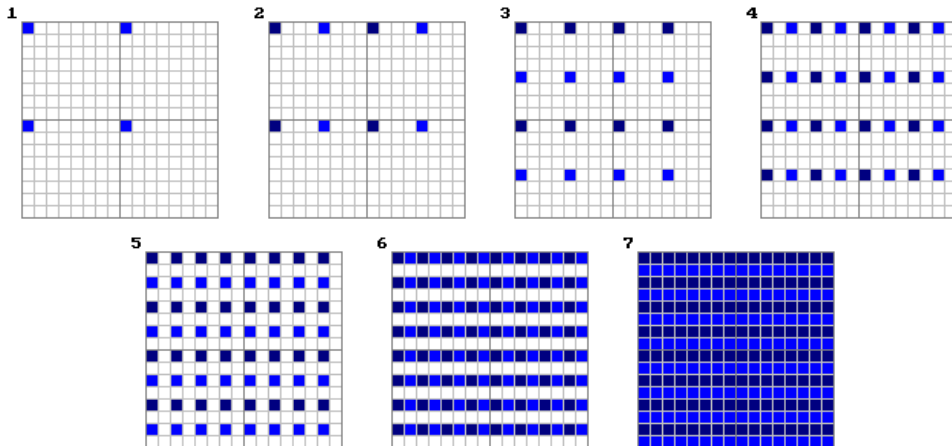


Animée
sur
AMeTICE

- Format Ouvert, GNU, lancé en 1995
- Se veut le remplaçant de GIF89
- Supporté par tous les navigateurs Web (y compris IE / Edge)
- Bibliothèque : <http://www.libpng.org/>
- Compression sans perte deflate (zlib)
- Permet : gris, couleurs, palette, Alpha-channel, affichage progressif pendant la lecture, etc.
- Donc plus général que GIF
- Variantes MNG (très peu répandue) ou A-PNG pour les animations

Décodage progressif : entrelacement Adam7

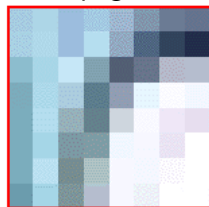
- Afficher progressivement une image au fur et à mesure du téléchargement (Web)
- Organiser les pixels en tableaux successifs de résolutions croissantes représentant une organisation entrelacée des pixels
- Interpoler pour l'affichage les pixels temporairement manquants



- Normalisé ISO (10918-1) en 1994
- Photo numérique ; mal adapté aux schémas très nets et/ou ayant peu de couleurs
- Codage avec perte des détails peu perceptibles par l'œil humain
- Compression élevée ; compromis de qualité contrôlée par l'utilisateur
- Traitements internes en YCbCr (Luminance + chrominance)
- Transformation (quasi)-réversible
- Bibliothèque libjpeg du Independent JPEG Group :
<http://www.ijg.org/>

Joint Photographic Experts Group (JPEG) (2/2)

- Passage en YCbCr
- Découpage en blocs 8×8



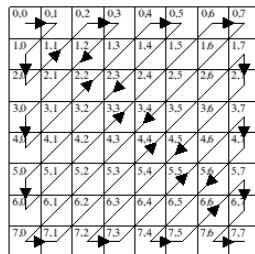
Block at 8×8 pixels

40	38	45	40	43	54	60	58
39	36	44	32	47	69	77	85
50	40	25	54	66	60	33	32
57	38	38	66	47	11	2	5
59	36	47	62	24	2	9	11
58	41	55	53	6	4	10	1
58	33	57	39	3	5	4	2
64	44	54	35	3	7	3	3

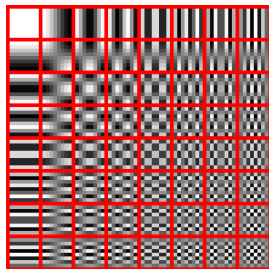
Color value matrix

44	-5	0	-4	0	-1	0	0
12	0	-3	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

DCT coefficients



- Sous-échantillonnage des couleurs (CbCr)
- Transformation en Cosinus Discrets (Discrete Cosine Transform, DCT)
- Parcours de la matrice de coefficients
- Compression RLE des 0 avec balise de fin lorsqu'il ne reste plus que des coefficients nuls
- Huffman final sur le résultat.



JPEG Qualité vs. compression (1/6)



Original
769 Ko



JPEG 99%
200 Ko (4 : 1)

JPEG Qualité vs. compression (2/6)



Original
769 Ko



JPEG 75%
52 Ko (15 : 1)

JPEG Qualité vs. compression (3/6)



Original
769 Ko



JPEG 50%
24 Ko (32 : 1)

JPEG Qualité vs. compression (4/6)



Original
769 Ko

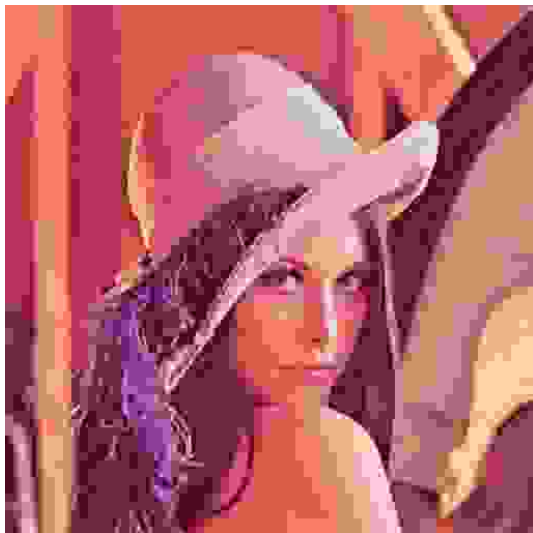


JPEG 25%
16 Ko (48 : 1)

JPEG Qualité vs. compression (5/6)



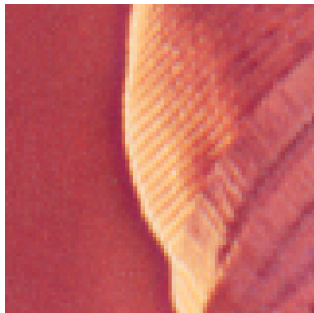
Original
769 Ko



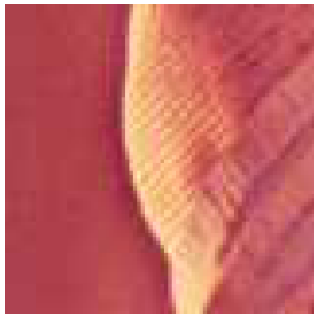
JPEG 1%
6 Ko (128 : 1)

JPEG Qualité vs. compression (6/6)

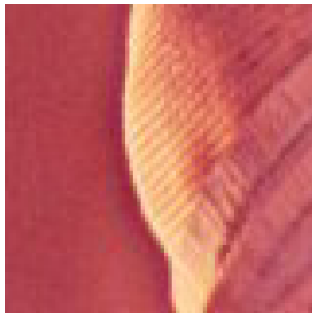
Original



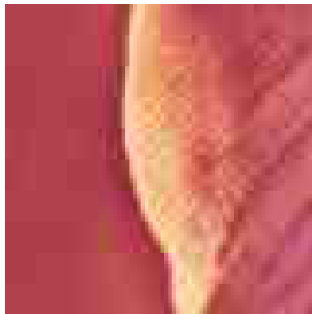
50%



75%



25%



- Bien meilleure qualité même à forte compression
- Absence de traitements par blocs
- Moins d'artefacts visibles
- Compression adaptative en fonction des zones dans l'image
- Peu de programmes grand public supportent JPEG2000
- Évolution future (JPEG20xx ?) possible et probable mais pas encore précisée avec les curvlets, bandlets, etc.
- Ne remplace malheureusement pas en pratique JPEG qui est très (trop ?) bien implanté !
- Les professionnels préfèrent les formats bruts sans perte (RAW ou DNG) accompagnés d'un « développement » numérique et délaissent les formats avec perte.

JPEG 2000 Qualité vs. compression (1/4)

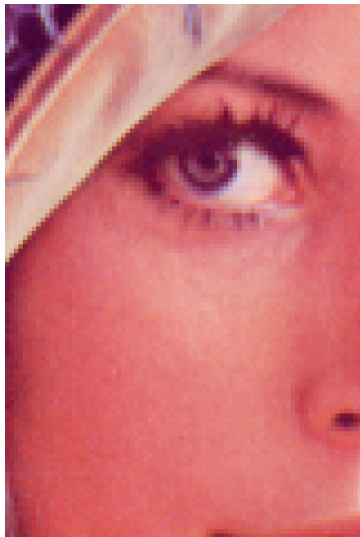


JPEG
29 Ko (26 : 1)

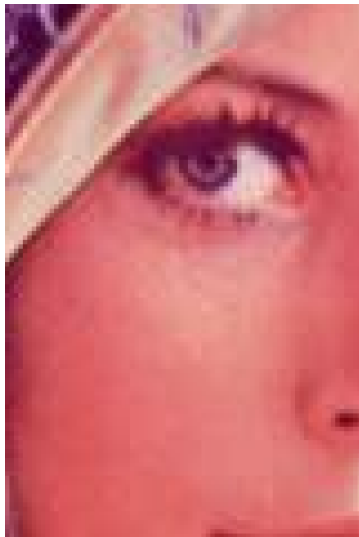


JPEG2000
29 Ko (26 : 1)

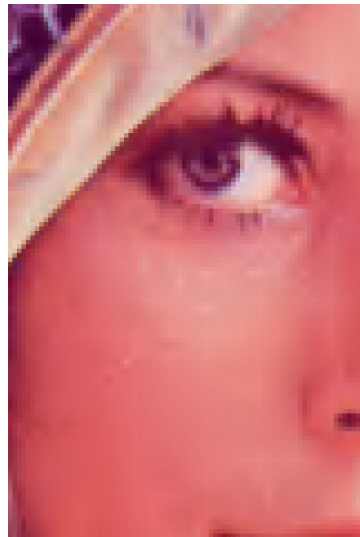
JPEG 2000 Qualité vs. compression (2/4)



Original

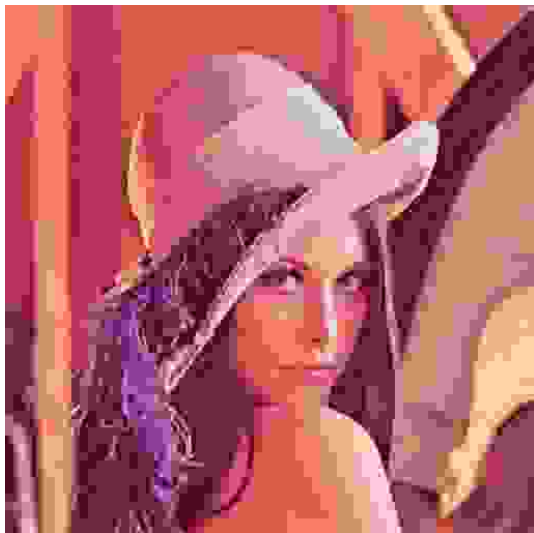


JPEG

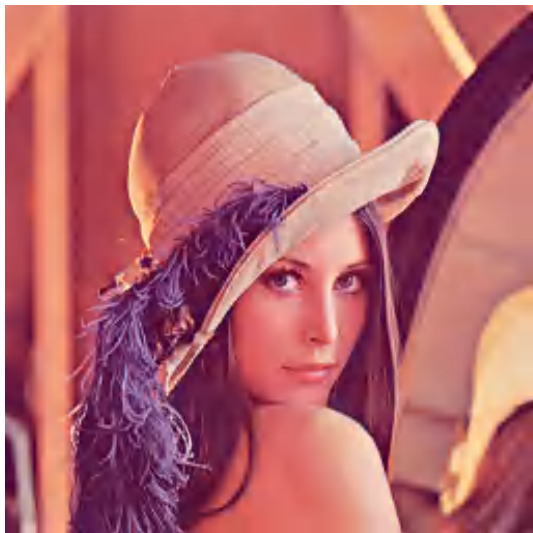


JPEG2000

JPEG 2000 Qualité vs. compression (3/4)



JPEG
6 Ko (128 : 1)

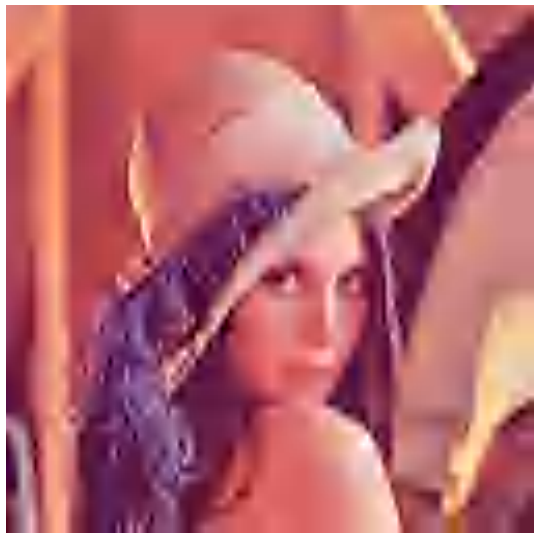


JPEG2000
6 Ko (128 : 1)



JPEG

...



JPEG2000

1,04 Ko (740 : 1)

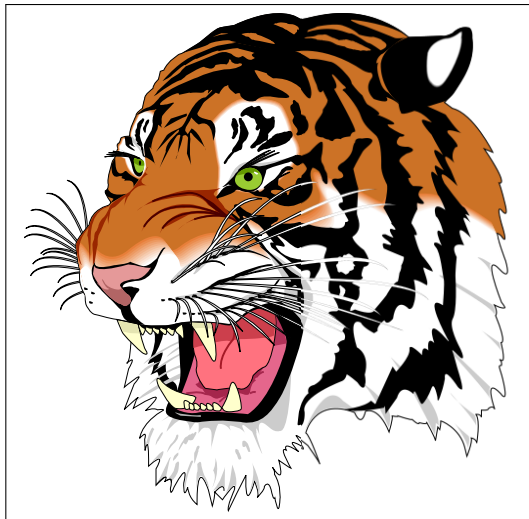
Tagged Image File Format (TIFF)

- (Actuellement) Adobe (gère les sous-formats)
- Très général :
 - fax, dessin au trait,..., photo numérique, etc.
 - sans compression, sans perte, avec perte JPEG, etc.
 - une ou plusieurs images, etc.
 - une ou plusieurs couches
 - palette(s) ou non
- Impossible de lire toutes les variantes !
- Format d'échange accepté par tout logiciel sérieux de traitement d'image.
- Meta-format encapsulant d'autres (JPEG par exemple)

- Exchangeable Image File Format
 - Variation de JPEG et TIFF d'origine Japonaise
 - Destiné à uniformiser le stockage des prises de vues issues d'appareils photos numériques
 - Permet de stocker des informations : date/heure, ouverture, focale, vignette, temps de pose, géolocalisation GPS, etc.
- International Press Telecommunications Council
 - Autre forme de méta-données
 - Origine en 1979 puis 91 ; perçe en 1994 (Adobe) pour JPEG/JFIF et TIFF
- Si vous avez un appareil photo numérique (ou un smartphone) jetez un coup d'œil à vos EXIFs !

Images vectorielles : principe

- Stocker la façon de dessiner plutôt que le résultat
- Stocker non pas des pixels mais des primitives : segments, formes, courbes, etc.
- Qualité indépendante de la résolution !
- Souvent très compact en taille de données (PDF : 37 Ko $\leq$ PNG : 156 Ko)
- Nécessite de tracer l'ensemble des primitives à chaque rendu
- Parfait pour tout ce qui est symbolique (texte, schémas, etc.)
- Inadapté pour une image du monde réel



Pour pouvoir zoomer fortement avec evince, penser à taper « `gsettings set org.gnome.Evince page-cache-size 500` »

- PostScript type 1 et type 3
 - Courbes de Bézier cubiques
- TrueType Fonts (.TTF)
 - Apple puis MicroSoft
 - Courbes quadratiques et segments droits
- Conversion sans perte possible TTF -> PS1 mais pas l'inverse du fait du degré des courbes !
- Existe aussi polices bitmap pour affichage rapide sur écrans mais inadaptées à l'impression papier (.FON)
- Très gros marché ! Une police peut coûter très cher : exemple du Times et du Monde

- XML → structure d'arbre
- Possibilité d'inclusion à d'autres fichiers XML, en particulier XHTML
- Spécification du W3C
- Gère l'alpha-channel, les dégradés, etc.
- Gère l'animation via JavaScript → remplace Macromedia Flash
- Les icônes (Gnome, KDE, ...) sont maintenant très souvent en SVG.
- Quelques logiciels : Sodipodi, Inkscape, Illustrator, OpenOffice Draw (export), etc.

<http://www.openclipart.org/>

PostScript

- Édition professionnelle
- Langage de description de page ; langage de programmation en polonaise inversée
- Interprété par les imprimantes haut de gamme (laser)
- Level1 (1984), Level2 (1991), Level3 (1997)



Adobe® PostScript® 3™

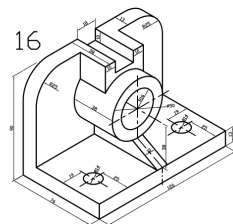
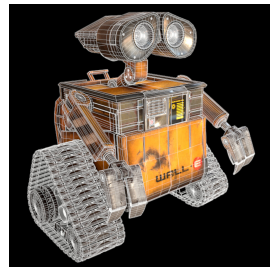
Portable Document Format (PDF)

- Successeur de PostScript plutôt destiné aux documents complexes et structurés
- Normalisé ISO en 2008 ; format ouvert
- 1.0 (1993), 1.1 (1994), 1.2 (1996), 1.3 (1999), 1.4 (2001), 1.5 (2003), 1.6 (2005), 1.7 (2006), 1.7 Extension Level 3 (2008), 1.7 Extension Level 5 (2009), 2.0 (2017)
- Incorporation des polices ; Compression de données ; Possibilité d'avoir des formulaires ; Liens URL ; Cryptage et protection



- Un système gratuit et éprouvé de composition de document
- Le système gère la composition typographique : il sépare le texte et la mise en forme qui en sera faite en suivant un style typographique de document.
- WYS(ee)IWYG $\neq$ WYW(ant)IWYG !
- Imbattable pour faire un beau document, en particulier s'il contient des formules mathématiques, des algorithmes, etc.
- Basé sur TeX de Donald Knuth (1978), amélioré par Leslie Lamport (1984)
- Langage de programmation à balises un peu comparable à HTML ; « Compilateur » pour produire le document (tex, latex, pdflatex, lualatex, etc.)
- METAFONT : langage et interprète pour la description de polices de caractères

- Même principe qu'une image vectorielle mais généralisée en 3D à des surfaces triangulées, Bezier, NURBS, etc.
- Des besoins différents (synthèse d'image ou usinage / impression 3D) conduisent à des procédés différents.
- Intègrent les textures, l'animation, ...
- Synthèse d'image : Lightwave Object (.LWO), PovRay (format texte de description de scène proche d'un langage de programmation), 3D Studio (.obj), 3D Studio Max (.max)
- Les formats gagnent en complexité et deviennent difficiles à lire !
- Fabrication, DAO/CAO : AutoCAD (.dxf, .dwg)
- Web et Réalité Virtuelle : Virtual Reality Markup Language (1994 à 1997 ; SGI puis ISO ; .wrl ou .vrml) ; est plus ou moins remplacé par X3D (W3C) mais il y a des tentatives divergentes : U3D (Intel) ; XAML (MicroSoft) ; 3D XML (Dassault Systèmes)



Des questions ?

